

BÀI KIỂM TRA NĂNG LỰC LẬP TRÌNH NHÚNG

MOTOR CONTROL – SỐ 1

Tăng lái cầu H 3 pha – Complementary PWM – Dead-time – Đồng bộ hóa lấy mẫu ADC

Vi điều khiển: STM32G474RET6 (LQFP64) | f_CLK = 170 MHz | Công cụ: STM32CubeMX + Keil MDK-ARM (HAL)

PHẦN 1 — BẢNG CHÂN & CẤU HÌNH PHẦN CỨNG CUỐI CÙNG

1.1. Bảng phân công chân (Pinout) — STM32G474RET6, LQFP64 (Chọn SMT32G474RET6 để đọc datasheet - CubeMX để đọc chân)

Tín hiệu	Chân	AF / Mode	Chức năng
TIM1_CH1	PC0	AF2, PP	PWM Phase A – High side
TIM1_CH2	PC1	AF2, PP	PWM Phase B – High side
TIM1_CH3	PC2	AF2, PP	PWM Phase C – High side
TIM1_CH1N	PC13	AF4, PP	PWM Phase A – Low side
TIM1_CH2N	PB0	AF6, PP	PWM Phase B – Low side
TIM1_CH3N	PB15	AF4, PP	PWM Phase C – Low side (đổi từ PB1 do PB1 dùng cho COMP1_INP)
TIM1_BKIN	PA6	AF6, Open-Drain (dự phòng — không dùng)	Chân TIM1_BKIN vật lý — KHÔNG dùng trong thiết kế này. COMP1 được route thẳng vào Break logic nội bộ của TIM1 qua thanh ghi TIM1_AF1 (BKCOMP1E, xem mục 1.4 & Phần 3), không đi qua bất kỳ chân GPIO nào. Giữ hàng này chỉ để tham khảo vị trí chân dự phòng nếu sau này cần đưa BKIN ra ngoài bằng phần cứng rời.
ADC1_IN1	PA0	Analog	Đọc dòng Shunt Phase A (Injected)
ADC1_IN2	PA1	Analog	Đọc dòng Shunt Phase B (Injected)

Tín hiệu	Chân	AF / Mode	Chức năng
ADC1_IN3	PA2	Analog	Đọc dòng Shunt Phase C (Injected)
COMP1_INP	PB1	Analog	Input dương Comparator1 – tín hiệu shunt tổng/quá dòng

1.2. Cấu hình TIM1 — Advanced Timer 3 pha bổ sung

Thông số	Giá trị	Ghi chú
Counter Mode	Center-Aligned Mode	Bắt buộc theo đề — dạng sóng đối xứng qua tâm
Clock Source	Internal Clock (170 MHz)	
Prescaler (PSC)	0	Không chia tần số
Auto-Reload (ARR)	4249	Diễn giải ngay bên dưới bảng
PWM Mode	PWM Mode 1 (OCxM = 110)	CH1/CH2/CH3
Dead Time	1.2 μ s (DTG = 0xA6)	Diễn giải chi tiết ở Câu 1, Phần 2
BRK State	Enable, Polarity High	Nguồn: COMP1 (Digital Input tắt)
Automatic Output Enable (AOE)	Disable	Yêu cầu bật lại PWM thủ công sau sự cố — an toàn hơn
TRGO (Update Event)	Update Event	Kích hoạt Injected ADC tại Underflow (đáy chu kỳ)
TRGO2 (Update Event)	Update Event	Dự phòng / đồng bộ ngoại vi khác nếu cần

Diễn giải giá trị ARR (Auto-Reload Register)

Ở chế độ Center-Aligned (đếm lên rồi đếm xuống), bộ đếm đi từ 0 → ARR (đếm lên, mất ARR xung clock) rồi từ ARR → 0 (đếm xuống, mất thêm ARR xung clock). Một chu kỳ PWM hoàn chỉnh do đó tương ứng $2 \times \text{ARR}$ xung clock Timer:

$$T_{\text{pwm}} = 2 \times \text{ARR} \times T_{\text{clk}}$$

$$\Rightarrow \text{ARR} = T_{\text{pwm}} / (2 \times T_{\text{clk}}) = f_{\text{clk}} / (2 \times f_{\text{pwm}})$$

$$f_{\text{clk}} = 170,000,000 \text{ Hz} ; f_{\text{pwm}} = 20,000 \text{ Hz}$$

$$\text{arr_val} = 170,000,000 / (2 \times 20,000) = 170,000,000 / 40,000 = 4250$$

Đây đúng là công thức đã có sẵn trong khung code mẫu (skeleton) của đề bài. Tuy nhiên biến trung gian `arr_val = 4250` chưa phải giá trị nạp trực tiếp vào thanh ghi — đề bài quy ước thanh ghi `TIM1->ARR` được nạp bằng `(arr_val - 1)`, tương tự cách tính ARR quen thuộc trong chế độ Edge-Aligned (đảm bảo đúng số mức đếm mong muốn, tránh lệch 1 bước do đếm bắt đầu từ 0):

$$\text{TIM1->ARR} = \text{arr_val} - 1 = 4250 - 1 = 4249$$

Kiểm tra lại tần số PWM thực tế khi nạp `ARR = 4249` vào thanh ghi:

$$T_{\text{pwm_thực}} = 2 \times \text{ARR} \times T_{\text{clk}} = 2 \times 4249 \times 5.882 \text{ ns} = 49,994.7 \text{ ns} \approx 49.995 \mu\text{s}$$

$$f_{\text{pwm_thực}} = 1 / 49.995 \mu\text{s} \approx 20,002 \text{ Hz} \text{ (lệch } \approx +0.01\% \text{ so với } 20 \text{ kHz danh định)}$$

Sai số $\sim 0.01\%$ này phát sinh do phải làm tròn ARR về số nguyên (4249 thay vì 4250 theo đúng công thức lý thuyết); đây là sai số làm tròn không tránh khỏi khi lượng tử hóa theo xung clock rời rạc, hoàn toàn nằm trong dung sai chấp nhận được của hệ thống điều khiển động cơ thực tế và không ảnh hưởng đến tính đúng đắn của thiết kế.

1.3. Cấu hình ADC1 — Injected Group (đọc dòng 3 pha)

Thông số	Giá trị
Enable Injected Conversions	Enable
Number Of Conversions	3
External Trigger Source	Timer 1 Trigger Out event (TRGO)
External Trigger Edge	Rising edge
Rank 1 / 2 / 3	Channel 1 (PA0) / Channel 2 (PA1) / Channel 3 (PA2)
Sampling Time (mỗi kênh)	2.5 Cycles

Thông số	Giá trị
Offset	No offset
Regular Group	Software start (không dùng, tránh xung đột trigger với Injected)

1.4. Cấu hình COMP1 — Bảo vệ quá dòng phần cứng

Thông số	Giá trị
Input [+]	PB1 (tín hiệu đo được từ mạch shunt/khuếch đại)
Input [–]	1/2 Internal VRef (ngưỡng so sánh cố định)
Output routing	Nội bộ → TIM1_BKIN (không qua GPIO/CPU)
TIM1 BRK Source – COMP1	Enable, Polarity High

PHẦN 2 — CÂU HỎI GIẢI TRÌNH

Câu 1 — Tính toán Dead-time Register (DTG)

Với $f_{CLK} = 170 \text{ MHz}$, chu kỳ tích tắc:

$$T_{clk} = 1 / 170,000,000 = 5.882 \text{ ns}$$

STM32 chia trường DTG[7:0] trong TIM1->BDTR thành 4 dải (range) theo 3 bit cao nhất DTG[7:5], mỗi dải có công thức và độ phân giải khác nhau:

- DTG[7:5] = 0xx → $DT = DTG[7:0] \times T_{clk}$ (độ phân giải $1 \times T_{clk}$, tối đa $127 \times T_{clk}$)
- DTG[7:5] = 10x → $DT = (64 + DTG[5:0]) \times 2 \times T_{clk}$ (độ phân giải $2 \times T_{clk}$)
- DTG[7:5] = 110 → $DT = (32 + DTG[4:0]) \times 8 \times T_{clk}$ (độ phân giải $8 \times T_{clk}$)
- DTG[7:5] = 111 → $DT = (32 + DTG[4:0]) \times 16 \times T_{clk}$ (độ phân giải $16 \times T_{clk}$)

Mục tiêu $DT = 1200 \text{ ns}$. Thử dải 1 (0xx): $DTG_{max} = 127 \times 5.882 \approx 747 \text{ ns}$ — không đủ, phải sang dải 2 (10x):

$$1200 = (64 + DTG[5:0]) \times 2 \times 5.882$$

$$64 + DTG[5:0] = 1200 / (2 \times 5.882) = 102.0$$

$$DTG[5:0] = 102 - 64 = 38 = 0b100110 \text{ (6 bit thấp)}$$

Mà ta có 2 bit cao Prefix: 10 (để chọn vùng thang đo x2)

$$\rightarrow DTG[7:0] = 10_100110b = 0xA6 = 166 \text{ (decimal)}$$

$$\rightarrow DT_{thực_tế} = 102 \times 2 \times 5.882 \text{ ns} = 1200.0 \text{ ns} \text{ (chính xác tuyệt đối)}$$

CubeMX tự tính và điền giá trị này khi nhập Dead Time = 1.2 μs trong giao diện Parameter Settings của TIM1; giá trị sinh ra trong code là:

```
sBreakDeadTimeConfig.DeadTime = 0xA6; // = 166
```

Câu 2 — Dạng sóng PWM & Bù Dead-time (Dead-time Compensation)

Quy ước chiều dòng điện dùng trong phân tích: $I_{phase} > 0$ nghĩa là **dòng điện chạy từ nhánh cầu ra tải** (ra cuộn dây động cơ); $I_{phase} < 0$ nghĩa là **dòng điện chạy từ tải về nhánh cầu**. Đây là quy ước phổ biến nhất trong các tài liệu về dead-time compensation; nếu quy ước ngược lại được sử dụng, kết luận về chiều tăng/giảm điện áp bên dưới sẽ đảo dấu tương ứng.

Dead-time là khoảng thời gian cả 2 MOSFET (High-side và Low-side) trên cùng 1 nhánh cầu đều bị khóa (OFF) để tránh ngắn mạch dọc. Trong khoảng thời gian này, điện áp thực tế đặt lên tải không

được quyết định bởi Gate Driver mà bởi chiều dòng điện pha đang chạy qua diode thân (body diode) của MOSFET:

- Khi $I_{\text{phase}} > 0$ (dòng chạy ra khỏi pha, từ nguồn qua tải): trong khoảng dead-time, dòng buộc phải tiếp tục chạy qua diode thân của MOSFET Low-side (freewheeling qua Low-side). Kết quả là điện áp đầu ra trong khoảng dead-time bị kéo xuống thấp hơn giá trị lý tưởng — điện áp trung bình thực tế THẤP HƠN so với giá trị duty cycle đã tính toán.
- Khi $I_{\text{phase}} < 0$ (dòng chạy vào pha, từ tải về nguồn): dòng freewheeling qua diode thân của MOSFET High-side. Điện áp đầu ra trong khoảng dead-time bị kéo lên cao hơn giá trị lý tưởng — điện áp trung bình thực tế CAO HƠN so với giá trị duty cycle đã tính toán.

Sai lệch điện áp này (ΔV) tỷ lệ thuận với tần số PWM và thời gian dead-time, không phụ thuộc độ lớn dòng điện (chỉ phụ thuộc dấu/chiều dòng):

$$\Delta V \approx (T_{\text{dead}} / T_{\text{pwm}}) \times V_{\text{dc}} \times \text{sign}(I_{\text{phase}})$$

Đây chính là nguồn gốc của hiện tượng méo dạng sóng dòng điện gần điểm qua-không (zero-crossing distortion) thường thấy trong điều khiển FOC. Để bù (Dead-time Compensation), thuật toán điều khiển cần đo/ước lượng dấu của I_{phase} tại mỗi chu kỳ PWM (qua ADC Injected 3-shunt) rồi cộng/trừ thêm 1 lượng offset vào giá trị duty cycle tính toán từ vòng lặp dòng điện, theo đúng chiều ngược với sai lệch đã phân tích ở trên, nhằm bù lại phần điện áp bị mất/dư do dead-time gây ra.

Câu 3 — Thời điểm Trigger ADC (ADC Trigger Timing)

3.1. Vì sao đo dòng tại đáy (Bottom Peak) chính xác nhất?

Với PWM Center-aligned và điều chế 3-Shunt Low-side, tại thời điểm đáy chu kỳ (Timer đếm về 0, Underflow), cả 3 van Low-side đồng thời ở trạng thái dẫn (ON) trong một khoảng thời gian ngắn quanh điểm đó — đây là thời điểm DUY NHẤT trong toàn chu kỳ mà dòng điện qua CẢ 3 điện trở shunt đều phản ánh đúng dòng pha thực (không bị gián đoạn bởi chuyển mạch). Ngược lại, tại đỉnh chu kỳ (Top Peak), các van High-side mới là van dẫn chính, dòng qua shunt Low-side không ổn định hoặc bị ngắt hoàn toàn tùy theo trạng thái duty cycle của từng pha, khiến việc lấy mẫu tại đó không đáng tin cậy. Ngoài ra, đáy chu kỳ trùng với thời điểm xa nhất so với các sự kiện chuyển mạch (switching edges) ở 2 bên, giúp giảm nhiễu (switching noise/ringing) ảnh hưởng tới phép đo ADC.

3.2. Rủi ro khi Duty Cycle quá nhỏ ($T_{\text{on}} < 1.5 \mu\text{s}$)

Khi một pha có Duty Cycle rất nhỏ, thời gian van Low-side pha đó dẫn (gần với 100% chu kỳ vì Low-side dẫn khi High-side tắt) không phải là vấn đề — vấn đề ngược lại nằm ở pha có Duty Cycle rất LỚN (gần 100%), khi đó thời gian van Low-side dẫn lại rất ngắn, có thể ngắn hơn thời gian ADC cần để lấy mẫu ổn định (Sample time + Dead-time + thời gian ổn định tín hiệu). Nếu $T_{\text{on}}(\text{Low-}$

side) quá ngắn, ADC có thể lấy mẫu ngay trong lúc quá độ chuyển mạch (switching transient), cho kết quả sai lệch lớn.

Cách xử lý rủi ro này ở tầng Driver thường gồm các kỹ thuật sau:

- Giới hạn Duty Cycle tối đa (Duty Cycle Clamping): không cho phép duty cycle vượt quá một ngưỡng an toàn (ví dụ 97–98%), đảm bảo luôn còn đủ thời gian tối thiểu cho Low-side dẫn và ADC lấy mẫu ổn định.
- Kỹ thuật dịch pha / bơm xung tối thiểu (Minimum Pulse Width / Pulse Shifting): khi phát hiện 1 pha có T_{on} Low-side dưới ngưỡng an toàn, chủ động kéo dài xung đó lên mức tối thiểu cần thiết trong 1 chu kỳ, đồng thời bù lại (chèn lỗi có kiểm soát) ở chu kỳ điều chế để không làm lệch giá trị điện áp trung bình dài hạn.
- Chuyển sang phương pháp ước lượng dòng gián tiếp (Phase Reconstruction / Current Reconstruction): với những chu kỳ không lấy mẫu được đủ 3 pha, dùng định luật Kirchhoff (tổng 3 dòng pha = 0) để suy ra dòng pha còn thiếu từ 2 pha đã đo được, hoặc nội suy từ các chu kỳ lân cận.

Câu 4 — Bảo vệ ngắt khẩn cấp: Hardware Break (BKIN) vs Software Interrupt (EXTI)

Đối với bảo vệ chống ngắn mạch cầu H (Shoot-through), cơ chế Hardware Break bắt buộc phải dùng vì lý do độ trễ (latency) và tính toàn vẹn logic:

So sánh độ trễ (Latency)

Tiêu chí	Hardware Break (BKIN → BDTR)	Software Interrupt (EXTI → ISR)
Đường tín hiệu	Comparator → thẳng vào logic BDTR của Timer (mạch tổ hợp cứng)	Comparator/GPIO → EXTI → NVIC → CPU fetch/decode → nhảy vào ISR → ghi thanh ghi GPIO/Timer bằng lệnh phần mềm
Độ trễ điển hình	Dưới 100 ns (vài chu kỳ clock nội bộ của Timer, không qua CPU)	Hàng trăm ns đến vài μs, tùy tần số CPU, độ sâu ngắt lồng nhau (nested interrupt), thời gian lưu ngữ cảnh (context save/restore) và code trong ISR
Phụ thuộc tải CPU	Không — hoạt động độc lập hoàn toàn với lõi CPU, kể cả khi CPU	Có — nếu CPU đang xử lý ngắt ưu tiên cao hơn hoặc bị treo (deadlock,

Tiêu chí	Hardware Break (BKIN → BDTR)	Software Interrupt (EXTI → ISR)
	đang treo/bị lỗi (hang)	hard fault khác), phản ứng bị trì hoãn hoặc không xảy ra
Hành vi khi kích hoạt	Toàn bộ 6 chân CHx/CHxN tự động ép về trạng thái Idle/Inactive (theo cấu hình OSSR/OSSR) ngay trong 1 chu kỳ clock của Timer	Phải chờ ISR chạy xong các lệnh ghi thanh ghi để tắt từng chân — trong thời gian chờ đó, PWM vẫn đang xuất xung bình thường

Với dòng điện ngắn mạch có thể tăng tới hàng trăm A chỉ trong vài trăm ns đến vài μ s (do điện cảm ký sinh rất nhỏ của mạch cầu H), độ trễ cỡ μ s của phương án ngắt phần mềm là hoàn toàn không chấp nhận được — đủ để phá hủy MOSFET trước khi ISR kịp chạy. Cơ chế Hardware Break vì vậy được STM32 thiết kế như một khối logic tổ hợp chuyên dụng (Break logic trong TIM1/TIM8), tách biệt hoàn toàn khỏi luồng thực thi lệnh của CPU, đảm bảo thời gian phản ứng gần như tức thời và không bị ảnh hưởng bởi bất kỳ trạng thái nào của phần mềm đang chạy.

PHẦN 3 — MÃ NGUỒN HOÀN CHỈNH (CMSIS Register-level)

Phần này điền đầy đủ các TODO còn để trống trong khung code mẫu (skeleton) của đề bài, dùng thanh ghi CMSIS trực tiếp như đề yêu cầu. Các đoạn được đánh dấu “Bổ sung” là phần cần thiết để mạch chạy được trên phần cứng thật nhưng chưa có sẵn trong skeleton gốc (skeleton gốc chỉ minh họa các TODO chính, không đầy đủ toàn bộ chu trình khởi tạo).

```
static void TIM1_3Phase_GPIO_Init(void) {
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN | RCC_AHB2ENR_GPIOBEN |
RCC_AHB2ENR_GPIOCEN;

    /* PC0, PC1, PC2 = TIM1_CH1/CH2/CH3, AF2, Push-Pull */
    GPIOC->MODER &= ~(3u<<(0*2)) | (3u<<(1*2)) | (3u<<(2*2)));
    GPIOC->MODER |= ((2u<<(0*2)) | (2u<<(1*2)) | (2u<<(2*2))); /* Alternate Function */
    GPIOC->AFR[0] |= (2u<<(0*4)) | (2u<<(1*4)) | (2u<<(2*4))); /* AF2 */

    /* PC13 = TIM1_CH1N, AF4 */
    GPIOC->MODER &= ~(3u<<(13*2));
    GPIOC->MODER |= (2u<<(13*2));
    GPIOC->AFR[1] |= (4u<<((13-8)*4)); /* AF4 */

    /* PB0 = TIM1_CH2N, AF6 */
    GPIOB->MODER &= ~(3u<<(0*2));
    GPIOB->MODER |= (2u<<(0*2));
    GPIOB->AFR[0] |= (6u<<(0*4)); /* AF6 */

    /* PB15 = TIM1_CH3N, AF4 */
    GPIOB->MODER &= ~(3u<<(15*2));
    GPIOB->MODER |= (2u<<(15*2));
    GPIOB->AFR[1] |= (4u<<((15-8)*4)); /* AF4 */

    /* Tốc độ Low/Medium đã đủ cho ứng dụng 20 kHz; Push-Pull, No pull-up/down */
    /* (OTYPER giữ mặc định = 0 = Push-Pull, OSPEEDR/PUPDR giữ mặc định) */
}
```

3.2. Khối 1 — TIM1_3Phase_PWM_Init (hoàn chỉnh)

```
#include "stm32g4xx.h"

#define TIM1_CLK_FREQ    170000000UL
#define PWM_FREQ        20000UL    // 20 kHz
#define DEAD_TIME_DTG    0xA6U     // = 1.2 us (tính ở Phần 2, Câu 1)

/* ----- */
/* 0. GPIO cho 7 chân TIM1 */
/* ----- */
static void TIM1_3Phase_GPIO_Init(void) {
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN | RCC_AHB2ENR_GPIOBEN |
    RCC_AHB2ENR_GPIOCEN;

    /* PC0, PC1, PC2 = TIM1_CH1/CH2/CH3, AF2, Push-Pull */
    GPIOC->MODER &= ~(3u<<(0*2)) | (3u<<(1*2)) | (3u<<(2*2)));
    GPIOC->MODER |= ((2u<<(0*2)) | (2u<<(1*2)) | (2u<<(2*2))); /* Alternate Function */
    GPIOC->AFR[0] |= (2u<<(0*4)) | (2u<<(1*4)) | (2u<<(2*4)); /* AF2 */

    /* PC13 = TIM1_CH1N, AF4 */
    GPIOC->MODER &= ~(3u<<(13*2));
    GPIOC->MODER |= (2u<<(13*2));
    GPIOC->AFR[1] |= (4u<<((13-8)*4)); /* AF4 */

    /* PB0 = TIM1_CH2N, AF6 */
    GPIOB->MODER &= ~(3u<<(0*2));
    GPIOB->MODER |= (2u<<(0*2));
    GPIOB->AFR[0] |= (6u<<(0*4)); /* AF6 */

    /* PB15 = TIM1_CH3N, AF4 */
    GPIOB->MODER &= ~(3u<<(15*2));
    GPIOB->MODER |= (2u<<(15*2));
    GPIOB->AFR[1] |= (4u<<((15-8)*4)); /* AF4 */

    /* Tốc độ Low/Medium đã đủ cho ứng dụng 20 kHz; Push-Pull, No pull-up/down */
```

```

/* (OTYPER giữ mặc định = 0 = Push-Pull, OSPEEDR/PUPDR giữ mặc định) */
}

/* ----- */
/* Khối 1 — TIM1_3Phase_PWM_Init (hoàn chỉnh) */
/* ----- */

void TIM1_3Phase_PWM_Init(void) {
    /* 0. Bổ sung: bật clock GPIO + cấu hình AF cho 7 chân */
    TIM1_3Phase_GPIO_Init();

    /* 1. Bật Clock cho TIM1 */
    RCC->APB2ENR |= RCC_APB2ENR_TIM1EN;

    /* TODO 1: ARR cho chế độ Center-aligned (đã tính ở Phần 1.2) */
    uint32_t arr_val = (TIM1_CLK_FREQ / (2 * PWM_FREQ)); // = 4250
    TIM1->ARR = arr_val - 1; // = 4249
    TIM1->PSC = 0; // Không chia tần số

    /* Bổ sung: chọn Center-aligned Mode 1 (bắt buộc theo đề, chưa có trong skeleton) */
    TIM1->CR1 &= ~TIM_CR1_CMS;
    TIM1->CR1 |= TIM_CR1_CMS_0; // CMS = 01b: Center-aligned Mode 1

    /* TODO 2: PWM Mode 1 cho CH1, CH2, CH3 (giữ nguyên theo skeleton, đã đúng) */
    TIM1->CCMR1 |= (6 << TIM_CCMR1_OC1M_Pos) | TIM_CCMR1_OC1PE;
    TIM1->CCMR1 |= (6 << TIM_CCMR1_OC2M_Pos) | TIM_CCMR1_OC2PE;
    TIM1->CCMR2 |= (6 << TIM_CCMR2_OC3M_Pos) | TIM_CCMR2_OC3PE;
    /* Lưu ý: skeleton gốc của đề có 2 chỗ ghi nhầm tên field ở khối CCMR:
    (1) dòng CH2 ghi "TIM_CCMR2_OC2M_Pos"/"TIM_CCMR2_OC2PE" — sai, vì
    trường OC2M/OC2PE nằm trong thanh ghi CCMR1 (không phải CCMR2); đã
    sửa lại thành TIM_CCMR1_OC2M_Pos/TIM_CCMR1_OC2PE ở dòng CH2 phía trên.
    (2) dòng CH3 ghi "TIM_CCMR3_OC3PE" — tên field này không tồn tại
    (CH3 nằm trong thanh ghi CCMR2, không phải CCMR3; CCMR3 trên STM32G4
    chỉ dùng cho CH5/CH6). Đã sửa lại thành đúng tên TIM_CCMR2_OC3PE ở trên. */

    /* Bổ sung: bật output CHx/CHxN — bắt buộc để PWM thực sự xuất ra chân,

```

```

skeleton gốc thiếu bước này */
TIM1->CCER |= TIM_CCER_CC1E | TIM_CCER_CC1NE
        | TIM_CCER_CC2E | TIM_CCER_CC2NE
        | TIM_CCER_CC3E | TIM_CCER_CC3NE;

/* TODO 3: Nạp Dead-time Generator — DTG = 0xA6 cho đúng 1.2 us */
TIM1->BDTR &= ~TIM_BDTR_DTG;
TIM1->BDTR |= (DEAD_TIME_DTG << TIM_BDTR_DTG_Pos);

/* TODO 4: Break Input — dùng COMP1 nội bộ làm nguồn (không dùng chân
BKIN vật lý GPIO, đúng theo thiết kế đã chốt ở Phần 1.4) */
TIM1->BDTR |= TIM_BDTR_BKE;          // Enable Break input logic
TIM1->BDTR |= TIM_BDTR_BKP;          // Break polarity: Active-High
TIM1->AF1 |= TIM1_AF1_BKCOMP1E;      // Chọn output COMP1 làm nguồn Break
TIM1->AF1 |= TIM1_AF1_BKCOMP1P;      // Polarity Active-High cho nguồn COMP1
/* QUAN TRỌNG — khác với skeleton gốc: KHÔNG set TIM_BDTR_AOE.
Skeleton đề bài có gợi ý bật AOE (tự động bật lại PWM sau Break),
nhưng với ứng dụng bảo vệ quá dòng, để AOE=0 (Disable) để bắt buộc
phải reset/xác nhận an toàn bằng phần mềm trước khi cho PWM chạy
lại — tránh nguy cơ tự động chạy lại khi lỗi chưa được khắc phục. */

/* TODO 5: TRGO tại Update Event (Underflow) để đồng bộ Injected ADC */
TIM1->CR2 &= ~TIM_CR2_MMS;
TIM1->CR2 |= (0x2 << TIM_CR2_MMS_Pos);

/* Cho phép Main Output và khởi động Counter */
TIM1->BDTR |= TIM_BDTR_MOE;
TIM1->CR1 |= TIM_CR1_CEN;
}

/* ----- */
/* Khối 2 — ADC_Injected_Trigger_Config (hoàn chỉnh) */
/* Trình tự chuẩn RM0440: ADVREGEN -> chờ ổn định -> ADCAL -> ADEN */
/* trước khi cấu hình Injected sequence — skeleton gốc chưa đề cập, */
/* nếu bỏ qua ADC sẽ không hoạt động đúng trên phần cứng thật. */

```

```

/* ----- */
void ADC_Injected_Trigger_Config(void) {
    /* Bổ sung: bật clock ADC12 + GPIOA, cấu hình PA0/PA1/PA2 = Analog */
    RCC->AHB2ENR |= RCC_AHB2ENR_ADC12EN | RCC_AHB2ENR_GPIOAEN;
    GPIOA->MODER |= (3u<<(0*2)) | (3u<<(1*2)) | (3u<<(2*2)); // Analog mode

    /* Bổ sung: bật bộ điều áp nội ADC, chờ ổn định theo datasheet (~20 us) */
    ADC1->CR &= ~ADC_CR_DEEPPWD;
    ADC1->CR |= ADC_CR_ADVREGEN;
    for (volatile uint32_t i = 0; i < 3400; i++) { __NOP(); } // ~20us @170MHz

    /* Bổ sung: hiệu chuẩn Single-ended trước khi bật ADC */
    ADC1->CR &= ~ADC_CR_ADCALDIF;
    ADC1->CR |= ADC_CR_ADCAL;
    while (ADC1->CR & ADC_CR_ADCAL) { /* chờ hiệu chuẩn xong */ }

    /* Bổ sung: bật ADC, chờ cờ sẵn sàng ADRDY */
    ADC1->ISR |= ADC_ISR_ADRDY; // xóa cờ cũ (ghi 1 để clear)
    ADC1->CR |= ADC_CR_ADEN;
    while (!(ADC1->ISR & ADC_ISR_ADRDY)) { /* chờ ADC sẵn sàng */ }

    /* TODO 6: Cấu hình chuỗi Injected — 3 kênh, trigger = TIM1_TRGO, Rising edge */
    ADC1->JSQR = (2u << ADC_JSQR_JL_Pos) // JL = (3 kênh - 1) = 2
        | (0x0u << ADC_JSQR_JEXTSEL_Pos) // 0000 = TIM1_TRGO
        | (0x1u << ADC_JSQR_JEXTEN_Pos) // 01 = Rising edge
        | (1u << ADC_JSQR_JSQ1_Pos) // Rank1 = Channel 1 (PA0, Phase A)
        | (2u << ADC_JSQR_JSQ2_Pos) // Rank2 = Channel 2 (PA1, Phase B)
        | (3u << ADC_JSQR_JSQ3_Pos); // Rank3 = Channel 3 (PA2, Phase C)

    /* Sampling time = 2.5 cycles (SMP=000) cho kênh 1,2,3 — theo đúng cấu hình CubeMX */
    ADC1->SMPR1 &= ~(ADC_SMPR1_SMP1 | ADC_SMPR1_SMP2 | ADC_SMPR1_SMP3);

    /* Cho phép ngắt JEOC để đọc JDR1/JDR2/JDR3 trong ISR (không đọc trong while(1)) */
    ADC1->IER |= ADC_IER_JEOCIE;
    NVIC_SetPriority(ADC1_2_IRQn, 0); // Ưu tiên cao — xử lý ngay sau mỗi chu kỳ PWM
}

```

```

NVIC_EnableIRQ(ADC1_2_IRQn);

/* Không gọi ADC_INJ_Start bằng phần mềm — ADC sẽ tự khởi động chuyển đổi
   mỗi khi nhận cạnh lên từ TIM1_TRGO, đúng yêu cầu "không trigger bằng
   phần mềm" của đề bài. */
}
/* ----- */
/* BỔ SUNG — phần còn thiếu so với yêu cầu đề bài "bare-metal hoàn toàn": */
/* Clock Config 170MHz thuần CMSIS (không gọi HAL SystemClock_Config) */
/* HSI16 -> PLL -> SYSCCLK = 170MHz, PLLM=4, PLLN=85, PLLR=2 */
/* ----- */
void SystemClock_Config_BareMetal(void) {
    /* 1. Đảm bảo HSI16 đã bật và ổn định (mặc định bật sau reset) */
    RCC->CR |= RCC_CR_HSION;
    while (!(RCC->CR & RCC_CR_HSIRDY)) { }

    /* 2. Bật clock PWR, chuyển VOS sang Range 1 + Boost mode
       (bắt buộc để chạy > 150MHz trên STM32G4) */
    RCC->APB1ENR1 |= RCC_APB1ENR1_PWREN;
    PWR->CR1 &= ~PWR_CR1_VOS;
    PWR->CR1 |= PWR_CR1_VOS_0;          // VOS = 01: Range 1
    while (PWR->SR2 & PWR_SR2_VOSF) { } // chờ điện áp ổn định
    PWR->CR5 &= ~PWR_CR5_R1MODE;       // R1MODE=0: bật chế độ Boost

    /* 3. Flash Latency — PHẢI tăng trước khi tăng xung nhịp.
       LƯU Ý QUAN TRỌNG: đối chiếu lại bảng "Number of wait states"
       trong RM0440 (mục Flash, chế độ Boost VOS Range1) trước khi nạp —
       nếu chọn sai số wait-state, CPU fetch lệnh sai và có thể treo/crash. */
    FLASH->ACR &= ~FLASH_ACR_LATENCY;
    FLASH->ACR |= FLASH_ACR_LATENCY_4WS; // giá trị tham khảo cho 170MHz Boost
mode
    while ((FLASH->ACR & FLASH_ACR_LATENCY) != FLASH_ACR_LATENCY_4WS) { }

    /* 4. Cấu hình PLL: HSI16 /PLLM=4 = 4MHz -> xPLLN=85 = 340MHz VCO -> /PLLR=2 =
170MHz */

```

```

RCC->CR &= ~RCC_CR_PLLON;
while (RCC->CR & RCC_CR_PLLRDY) { } // chờ PLL tắt hẳn trước khi sửa cấu hình

RCC->PLLCFGR = 0;
RCC->PLLCFGR |= RCC_PLLCFGR_PLLSRC_HSI; // Nguồn PLL = HSI16
RCC->PLLCFGR |= (4UL - 1) << RCC_PLLCFGR_PLLM_Pos; // PLLM = 4
RCC->PLLCFGR |= 85UL << RCC_PLLCFGR_PLLN_Pos; // PLLN = 85
RCC->PLLCFGR |= 0UL << RCC_PLLCFGR_PLLR_Pos; // PLLR = 2 (mã 00b)
RCC->PLLCFGR |= RCC_PLLCFGR_PLLREN; // Bật ngõ ra PLLR (SYSCLK)

RCC->CR |= RCC_CR_PLLON;
while (!(RCC->CR & RCC_CR_PLLRDY)) { } // chờ PLL khóa (lock)

/* 5. Prescaler AHB/APB1/APB2 = /1 (giữ 170MHz cho toàn bộ bus) */
RCC->CFGR &= ~(RCC_CFGR_HPRE | RCC_CFGR_PPRE1 | RCC_CFGR_PPRE2);

/* 6. Chuyển SYSCLK sang PLL */
RCC->CFGR &= ~RCC_CFGR_SW;
RCC->CFGR |= RCC_CFGR_SW_PLL;
while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) { }

SystemCoreClockUpdate(); // cập nhật biến CMSIS SystemCoreClock
}

/* ----- */
/* BỔ SUNG — ISR đọc ADC Injected thuần thành ghi (không qua HAL) */
/* Thay thế ADC1_2_IRQHandler() do CubeMX sinh (gọi HAL_ADC_IRQHandler) */
/* ----- */
void ADC1_2_IRQHandler(void) {
    if (ADC1->ISR & ADC_ISR_JEOC) {
        uint16_t iA = (uint16_t)ADC1->JDR1; // Dòng Phase A
        uint16_t iB = (uint16_t)ADC1->JDR2; // Dòng Phase B
        uint16_t iC = (uint16_t)ADC1->JDR3; // Dòng Phase C

        /* TODO: xử lý FOC / kiểm tra ngưỡng quá dòng phần mềm (dự phòng) tại đây */
    }
}

```

```

    ADC1->ISR |= ADC_ISR_JEOC; // Xóa cờ (ghi 1 để clear)
}
}

/* ----- */
/* Ví dụ gọi trong main() — thứ tự bắt buộc: GPIO/TIM1 trước, ADC sau */
/* ----- */
int main(void) {
    SystemClock_Config_BareMetal(); // Bổ sung — thay HAL_Init()/SystemClock_Config() bằng
    bản CMSIS thuần thành ghi ở trên

    TIM1_3Phase_PWM_Init(); // Khối 1: PWM 3 pha + Dead-time + Break
    ADC_Injected_Trigger_Config(); // Khối 2: ADC Injected đồng bộ TRGO

    while (1) {
        /* Vòng lặp chính KHÔNG đọc ADC ở đây — dữ liệu dòng điện được
        xử lý trong ISR ADC1_2_IRQHandler() ngay khi JEOC xảy ra */
    }
}

```


— Hết —